TRANSPORT AND TELECOMMUNICATION INSTITUTE

ENGINEERING FACULTY

Individual project N2

Course: Individual project 2

Theme: **Hotel Management System**

Student: Arturs Vitols

St. code: 84129

Group: 4301BNA

Riga

2025

Contents

INTRODUCTION 4

1. TASK FOR THE PROJECT 5

2. REQUIREMENTS ANALYSIS AND TASK DEFINITION 6

2.1. Analysis of the subject are 6

2.2. Objects and attributes 6

2.3. Functional requirements 7

3. DESIGN 8

3.1. Navigation and User Interface Structure 8

3.2. Data Models and User-Defined Types 10

3.3. Data Storage Design 11

3.4. Class Diagram 13

3.5. Algorithms 14

4. DEVELOPMENT 15

4.1. Created user-defined types 15

4.2. Developed user interface 15

4.3. List of source files 18

4.4. List of entities. ERD 19

4.5. Class description and class diagram 19

4.6. Demonstration of application functionality 20

5. TESTING 23

5.1. How testing was organised 23

5.2. Scripts 23

5.3. Scenarios 23

5.4. Test cases 23

5.5. Test results 24

5.6. Found errors and corrections 24

CONCLUSIONS 25

REFERENCES 26

INTRODUCTION

The purpose of this course project is to design and implement a desktop-based Hotel Management System using object-oriented programming principles in C#. The application is intended to support basic hotel administration operations, including management of guests, rooms, and reservations, through a graphical user interface.

The system provides functionality for adding, editing, deleting, searching, and displaying data related to hotel operations. All data are stored locally in JSON files, ensuring persistence between application runs without the use of a database management system.

The project focuses on the practical application of object-oriented design concepts such as encapsulation, abstraction, and separation of concerns. The system architecture is divided into models, services, and user interface components, which improves code readability, maintainability, and scalability.

The developed Hotel Management System demonstrates how object-oriented programming techniques can be applied to solve real-world problems in software development. In addition, the application includes a statistics module that dynamically calculates aggregated information based on the current system data.

1. TASK FOR THE PROJECT

The main task of this course project is to design and implement a functional desktop application that automates key hotel management processes using object-oriented programming principles. To achieve this goal, the following tasks were defined:

• Analyse the hotel management domain and identify the main entities and their relationships.
• Design object-oriented models representing guests, rooms, and reservations.
• Develop a graphical user interface that allows users to manage system data efficiently.
• Implement functionality for adding, editing, deleting, searching, sorting, and displaying data.
• Ensure persistent storage of application data using local JSON files.
• Apply object-oriented programming concepts such as encapsulation, abstraction, and separation of concerns.
• Implement basic data validation and error handling to ensure data consistency.
• Develop a statistics module that provides aggregated information based on stored data.
• Test the application to verify the correctness and stability of all implemented features.

Completion of these tasks ensures that the developed application meets the functional and technical requirements of the course project and demonstrates practical application of object-oriented software development techniques.

2. REQUIREMENTS ANALYSIS AND TASK DEFINITION

2.1. Analysis of the subject are

The hotel industry requires efficient management of large amounts of guest-related data, such as reservations, room numbers, and stay periods. Traditionally, such data has been managed manually or using spreadsheets, which can lead to errors, delays, and inefficiency. A dedicated hotel management system allows automation of these processes, improving accuracy and reducing workload for hotel staff.

2.2. Objects and attributes

Based on the analysis of the subject area, three main objects were identified in the system:

- Guest
- Room
- Reservation

The Guest object represents a hotel guest and contains attributes such as:

- GuestID
- FirstName
- LastName
- CheckInDate
- CheckOutDate
- RoomNumber.

These attributes store personal information and accommodation details of the guest.

The Room object represents a hotel room and includes attributes such as:

- RoomNumber
- Type
- Status
- PricePerNight

This object is used to describe room characteristics and availability.

The Reservation object represents a booking made by a guest. It includes attributes such as:

- ReservationID

- GuestID
- RoomNumber
- CheckInDate
- CheckOutDate
- PaymentStatus

This object establishes a relationship between guests and rooms and stores information related to booking and payment.

2.3. Functional requirements

The system must provide functionality for managing hotel data through a graphical user interface. Users must be able to add, edit, delete, search, sort, and display information related to guests and reservations.

The system must ensure persistent storage of data so that all information is preserved between application runs. This requirement is satisfied through the use of local JSON files for data storage.

The application must perform basic validation of user input, including date validation and mandatory field checks, to prevent incorrect or inconsistent data from being stored.

In addition, the system must provide a statistics module that calculates aggregated information based on the stored data, such as the number of guests, reservations, and room occupancy status.

3. DESIGN

3.1. Navigation and User Interface Structure

The user interface of the Hotel Management System is designed to provide clear navigation and ease of use. The application follows a menu-based structure, where the main form serves as a central entry point to all system functionality.

The MainForm contains navigation controls that allow the user to access different functional modules of the system. From the main menu, the user can open separate forms for managing guests, rooms, reservations, and viewing statistical information. Each form is responsible for a specific set of operations related to its domain.

The Guests management form allows the user to add, edit, delete, search, and display guest information. The Rooms form provides an overview of hotel rooms, including room types, availability status, and pricing. The Reservations form enables the creation, modification, deletion, and searching of reservations, as well as tracking payment status. The Statistics form displays aggregated information calculated dynamically based on the current system data.

This navigation structure ensures that the user interface remains intuitive and modular. Separating functionality across multiple forms reduces interface complexity and improves usability by allowing users to focus on one task at a time.

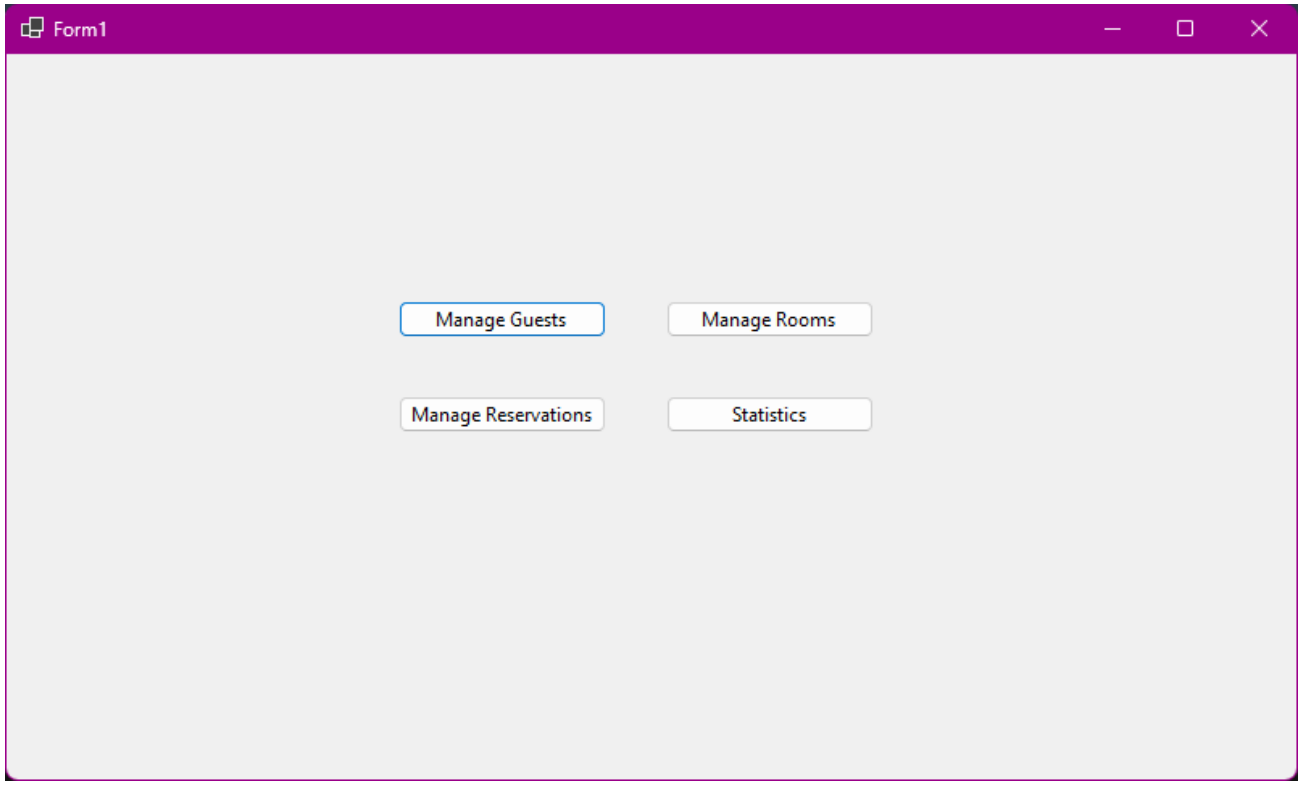

Fig. 1. Main menu

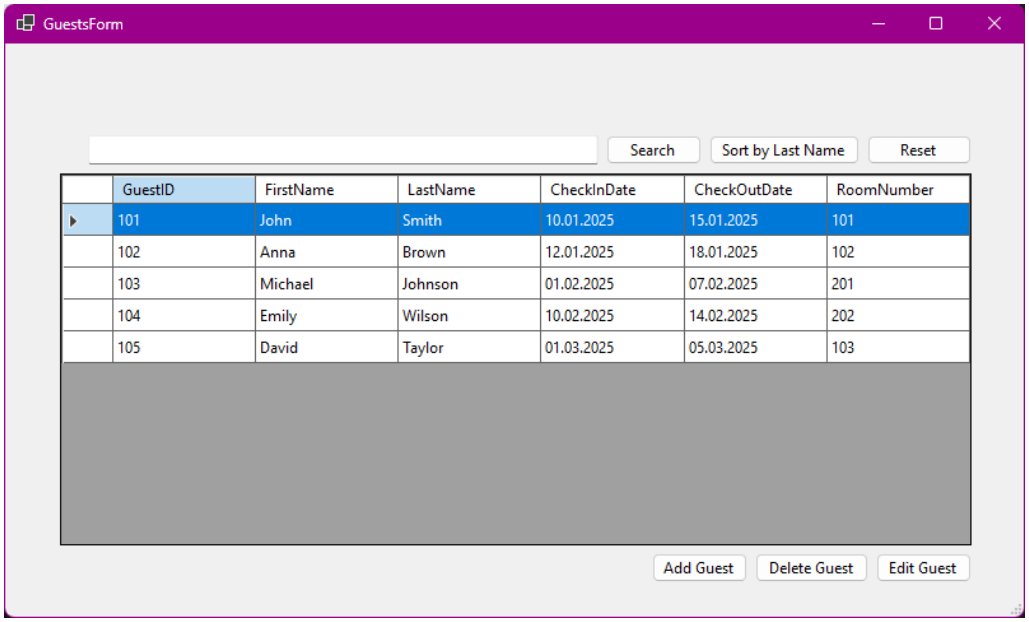

Fig. 2. Guests

3.2. Data Models and User-Defined Types

The Hotel Management System is based on a set of user-defined data models that represent the core entities of the hotel management domain. These models were identified during the analysis of the subject area and implemented using object-oriented programming principles.

Each data model is represented by a separate class and encapsulates the attributes relevant to a specific entity. This approach ensures clear separation of responsibilities and improves maintainability of the system. In addition, enumeration types are used to represent predefined sets of values, reducing the possibility of invalid data input.

The main data models used in the system are Guest, Room, and Reservation. Their attributes are presented in the tables below.

Table 1. Guest data model

Attribute	Type	Description
GuestID	int	Unique identifier of the guest
FirstName	string	Guest first name
LastName	string	Guest last name
CheckInDate	DateTime	Guest check-in date
CheckOutDate	DateTime	Guest check-out date
RoomNumber	int	Assigned room number

Table 2. Room data model

Attribute	Type	Description
RoomNumber	int	Unique room number
Type	RoomType	Type of room (Single, Double, Suite)
Status	RoomStatus	Availability status of the room
PricePerNight	decimal	Cost per night

Table 3. Reservation data model

Attribute	Type	Description
ReservationID	int	Unique reservation identifier
GuestID	int	Identifier of the related guest
RoomNumber	int	Reserved room number
CheckInDate	DateTime	Reservation check-in date
CheckOutDate	DateTime	Reservation check-out date
PaymentStatus	PaymentStatus	Current payment status

Enumeration types

To improve data consistency and code readability, several enumeration types are used in the system. These types define a fixed set of valid values and prevent incorrect assignments.

- RoomType: Single, Double, Suite
- RoomStatus: Available, Occupied, Maintenance
- PaymentStatus: Pending, Paid, Cancelled

3.3. Data Storage Design

The Hotel Management System uses file-based data storage to ensure persistence of application data between program executions. Instead of using a database management system, the application stores all data locally in JSON files. This approach was chosen due to its simplicity, transparency, and suitability for a small-scale desktop application.

Separate JSON files are used to store information related to different entities in the system. Guest data and reservation data are saved in dedicated files, which are automatically loaded when the application starts. Any changes made by the user, such as adding, editing, or deleting records, are immediately written back to the corresponding JSON file.

The JSON format provides a structured and human-readable representation of data, making it easy to inspect and modify if necessary. File input and output operations are handled by dedicated manager classes, which encapsulate all data access logic and prevent direct interaction between the user interface and storage mechanisms.

This storage design ensures data consistency, supports persistent data management, and follows the principle of separation of concerns by isolating data storage logic from other parts of the application.

Example of JSON file used for persistent data storage:

guests.json | reservations.json | MainForm.cs | MainForm.cs [Конструктор]
StatisticsForm.cs | StatisticsForm... [Конструктор]
Схема: <Схема не выбрана>

```
[
  {
    "GuestID": 101,
    "FirstName": "John",
    "LastName": "Smith",
    "CheckInDate": "2025-01-10T00:00:00",
    "CheckOutDate": "2025-01-15T00:00:00",
    "RoomNumber": 101
  },
  {
    "GuestID": 102,
    "FirstName": "Anna",
    "LastName": "Brown",
    "CheckInDate": "2025-01-12T00:00:00",
    "CheckOutDate": "2025-01-18T00:00:00",
    "RoomNumber": 102
  },
  {
    "GuestID": 103,
    "FirstName": "Michael",
    "LastName": "Johnson",
    "CheckInDate": "2025-02-01T00:00:00",
    "CheckOutDate": "2025-02-07T00:00:00",
    "RoomNumber": 201
  },
```

Fig. 3. guests.json

reservations.json | MainForm.cs | MainForm.cs [Конструктор] | StatisticsForm.cs
StatisticsForm... [Конструктор]
Схема: <Схема не выбрана>

```
[
  {
    "ReservationID": 1001,
    "GuestID": 101,
    "RoomNumber": 101,
    "CheckInDate": "2025-01-10T00:00:00",
    "CheckOutDate": "2025-01-15T00:00:00",
    "PaymentStatus": 1
  },
  {
    "ReservationID": 1002,
    "GuestID": 102,
    "RoomNumber": 102,
    "CheckInDate": "2025-01-12T00:00:00",
    "CheckOutDate": "2025-01-18T00:00:00",
    "PaymentStatus": 0
  },
  {
    "ReservationID": 1003,
    "GuestID": 103,
    "RoomNumber": 201,
    "CheckInDate": "2025-02-01T00:00:00",
    "CheckOutDate": "2025-02-07T00:00:00",
    "PaymentStatus": 1
  },
```

Fig. 4. reservations.json

3.4. Class Diagram

The class diagram of the Hotel Management System illustrates the main classes of the application and the relationships between them. The diagram reflects the object-oriented structure of the system and is based directly on the implemented code.

The core classes of the system include Guest, Room, and Reservation, which represent the main data models. These classes store the attributes related to guests, rooms, and reservations respectively. The Reservation class establishes a logical relationship between guests and rooms through identifiers, allowing the system to associate reservations with specific guests and room numbers.

Business logic is implemented through manager classes such as GuestManager and ReservationManager. These classes are responsible for performing operations on the corresponding data models, including adding, editing, deleting, searching, and retrieving records. The manager classes also handle data persistence by reading from and writing to JSON files.

The user interface layer consists of multiple Windows Forms, including MainForm, GuestsForm, RoomsForm, ReservationsForm, Add/Edit forms, and StatisticsForm. These forms interact with the manager classes to perform operations, but they do not directly access or manipulate stored data. This design ensures separation of concerns and improves maintainability.

Overall, the class diagram demonstrates a clear separation between data models, business logic, and user interface components, which is a key principle of object-oriented software design.

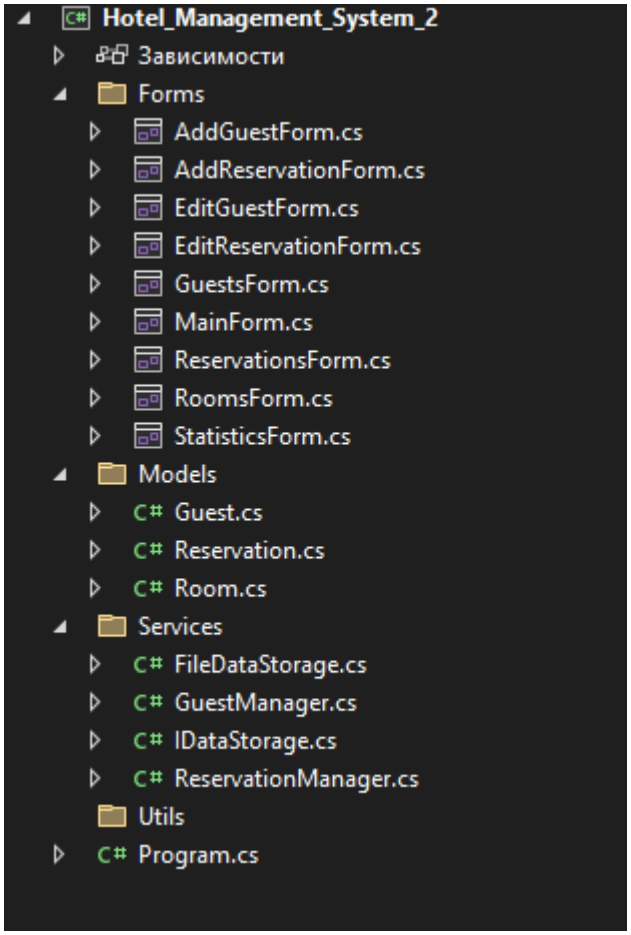

Fig. 5. Project structure reflecting class organization

3.5. Algorithms

The main algorithms implemented in the Hotel Management System are related to managing guests, rooms, and reservations. These algorithms define the sequence of actions performed by the system in response to user interactions through the graphical interface.

Add operation algorithm:

When the user selects the option to add a new record, the system opens a dedicated input form. The user enters the required data, which is then validated by the system. If the input data is valid, a new object is created and added to the corresponding collection. The updated data is immediately saved to a JSON file, and the user interface is refreshed to display the new record.

Edit operation algorithm:

To edit an existing record, the user selects an item from the data table and opens the edit form. The system loads the current data of the selected object into the input fields. After the user modifies the data and confirms the changes, the system validates the input and updates the

existing object. The modified data is then saved to the JSON file, and the displayed data is refreshed.

Delete operation algorithm:

When the delete operation is selected, the system identifies the chosen record and requests confirmation from the user. After confirmation, the record is removed from the collection, the updated data is saved to the JSON file, and the user interface is updated accordingly.

Search and sort algorithm:

The system allows users to search for records based on entered keywords. The search algorithm iterates through the stored data and filters records that match the search criteria. Sorting operations reorder the displayed data based on selected attributes, improving data accessibility and usability.

These algorithms ensure consistent handling of data across the system and support the functional requirements defined during the analysis stage.

4. DEVELOPMENT

4.1. Created user-defined types

In the development stage, user-defined types were implemented in the form of C# classes to represent the main entities of the system. These types were created according to the design specifications and are used throughout the application.

The following user-defined types were implemented:

- Guest
- Room
- Reservation

4.2. Developed user interface

The user interface of the application was developed using Windows Forms. All functional forms described in the design stage were fully implemented and connected to the business logic components.

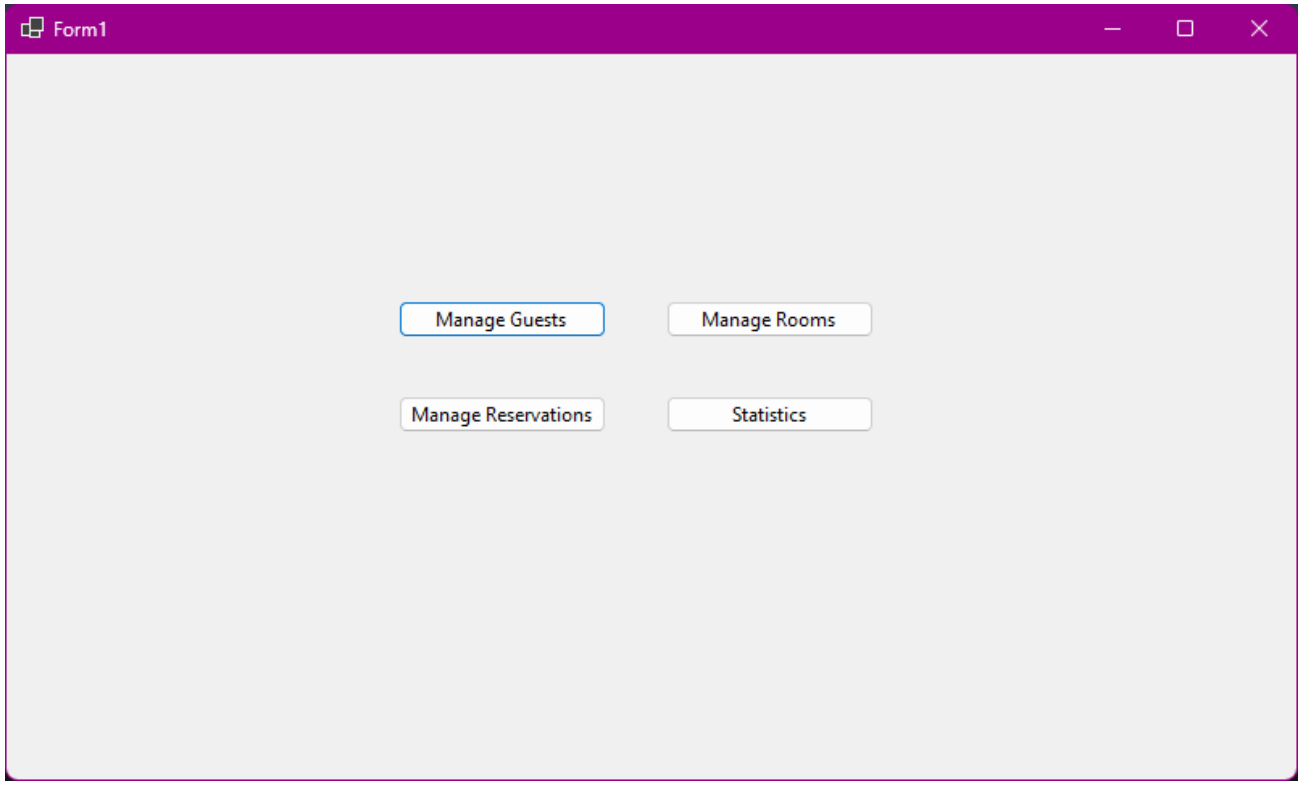

Fig.6. Main menu

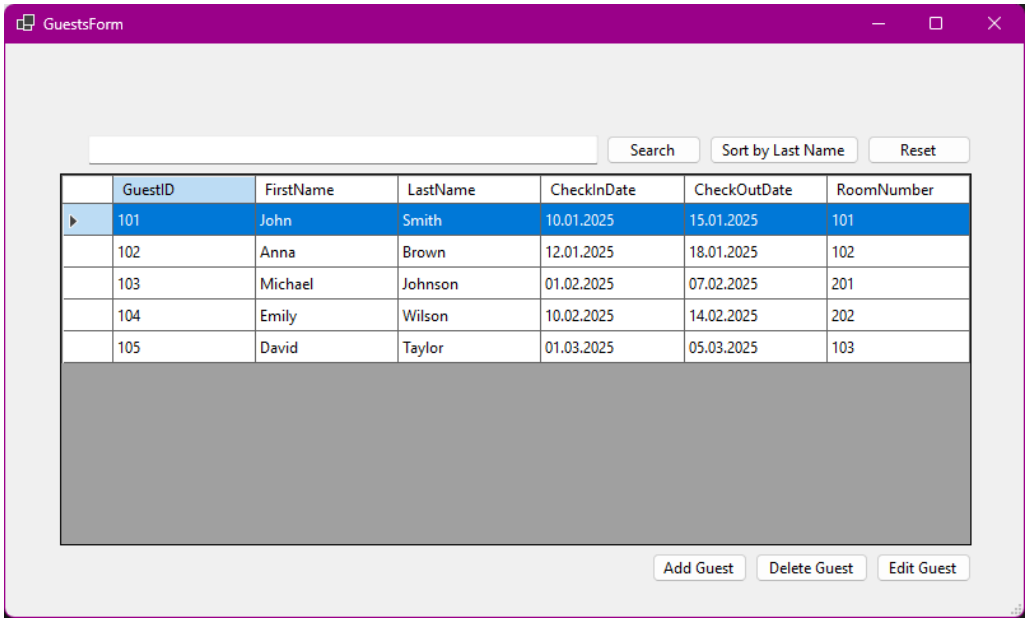

Fig.7. Guests management form

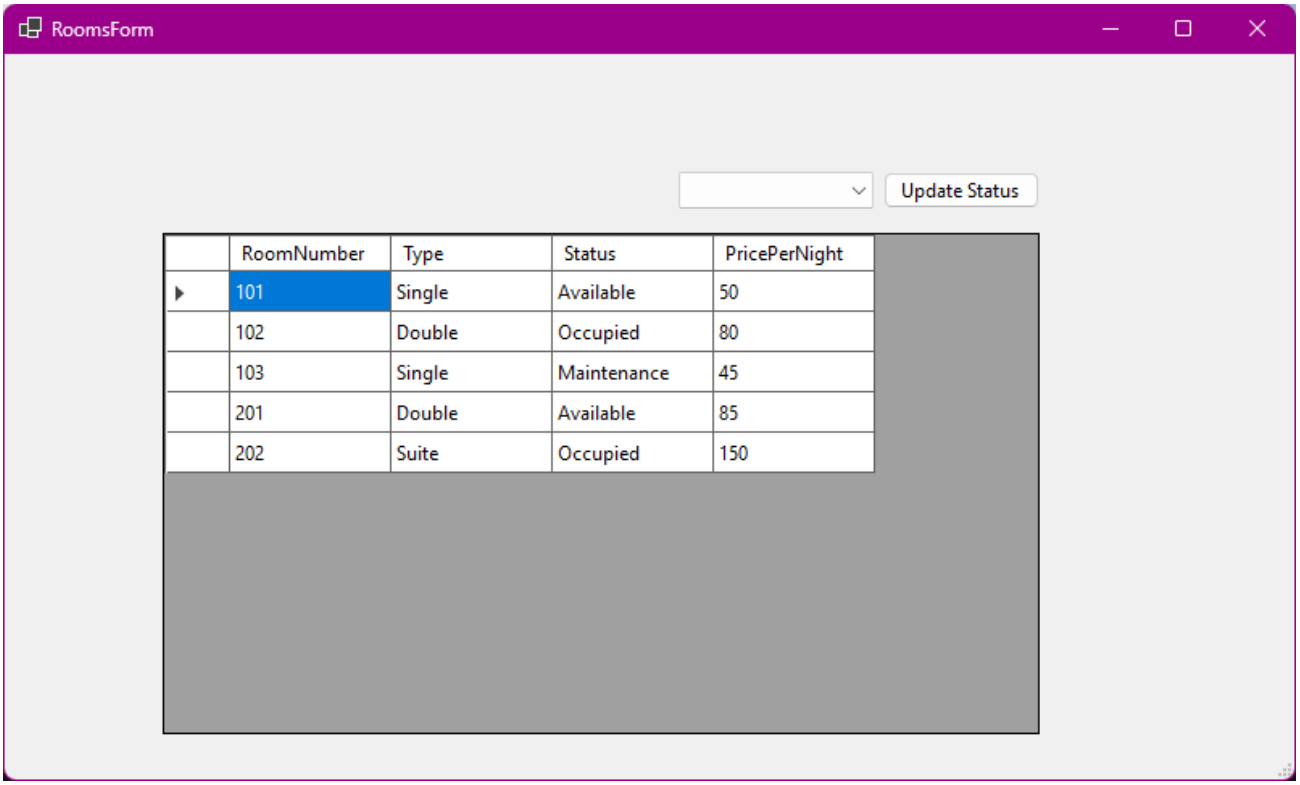

Fig.8. Rooms form

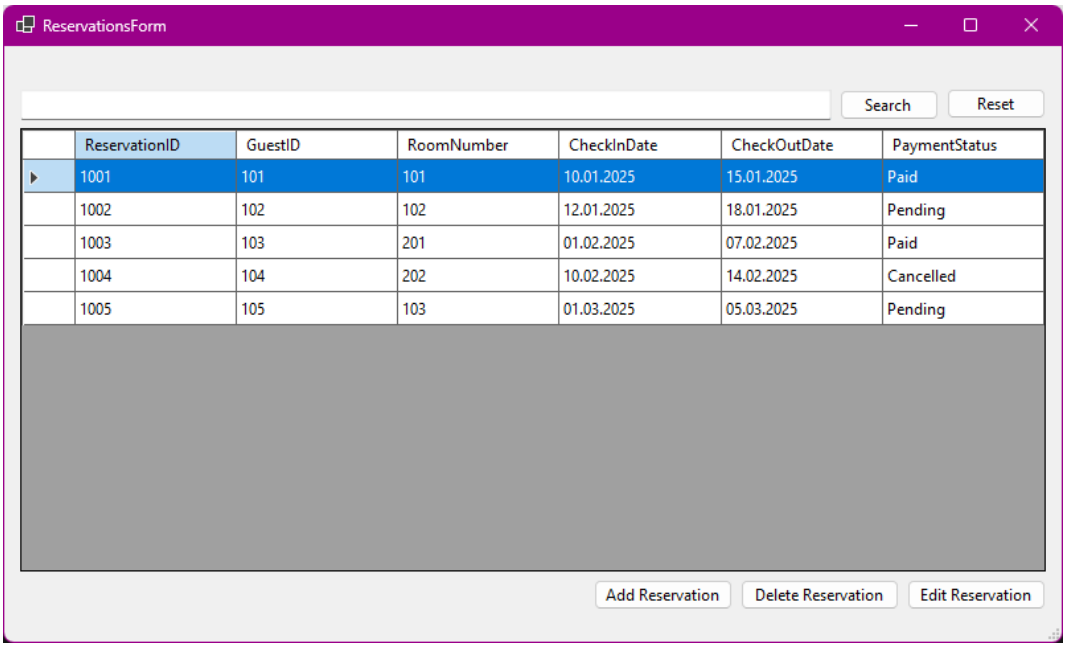

Fig.9. Reservations form

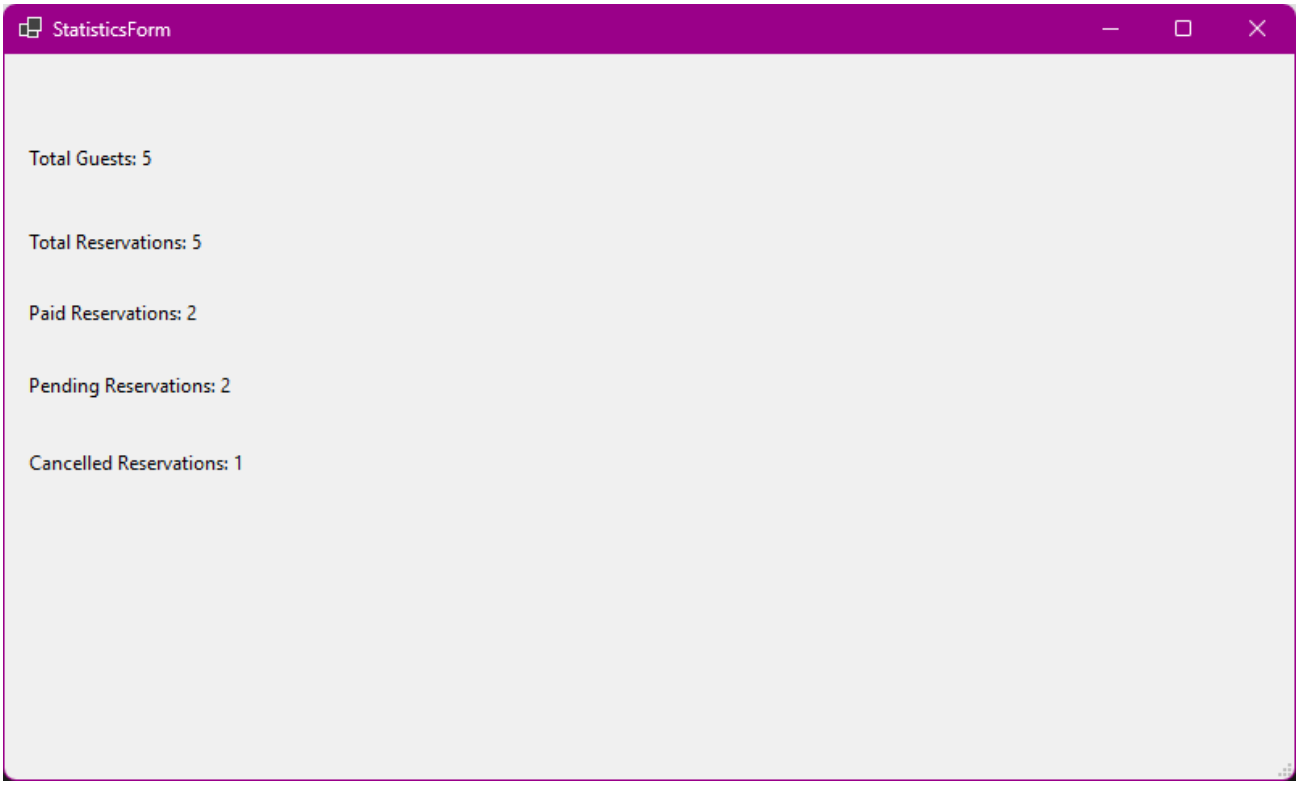

Fig.10. Statistics form

4.3. List of source files

The application consists of multiple source code files organised according to their functionality. The main source files of the project include:

- Program.cs
- MainForm.cs
- GuestsForm.cs
- RoomsForm.cs
- ReservationsForm.cs
- StatisticsForm.cs
- Guest.cs
- Room.cs
- Reservation.cs
- GuestManager.cs
- ReservationManager.cs

4.4. List of entities. ERD

The main entities of the system were identified during the analysis stage and implemented during development. The logical relationship between these entities is shown in the entity-relationship diagram.

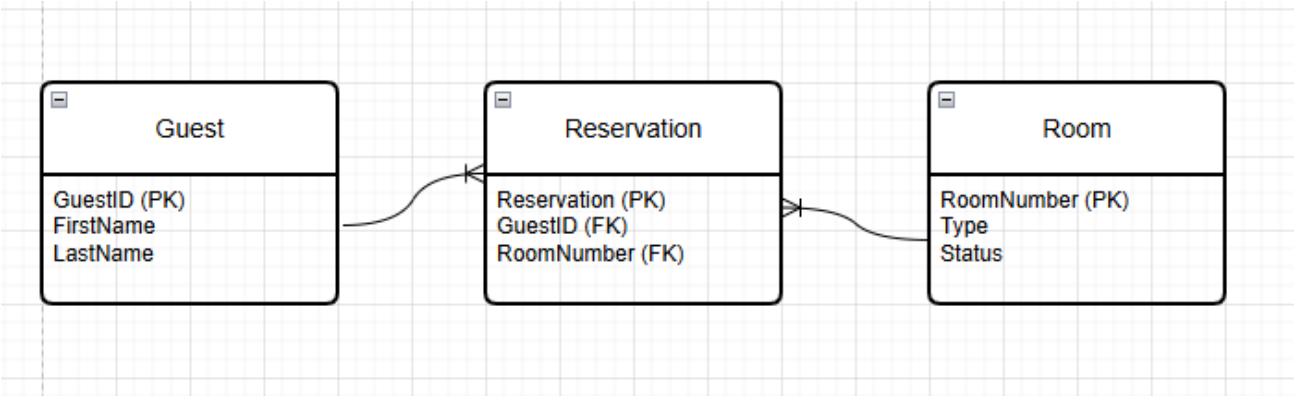

Fig.11. ERD

4.5. Class description and class diagram

All system classes were implemented according to the designed class structure. The class diagram reflects the relationships between data models, manager classes, and user interface forms.

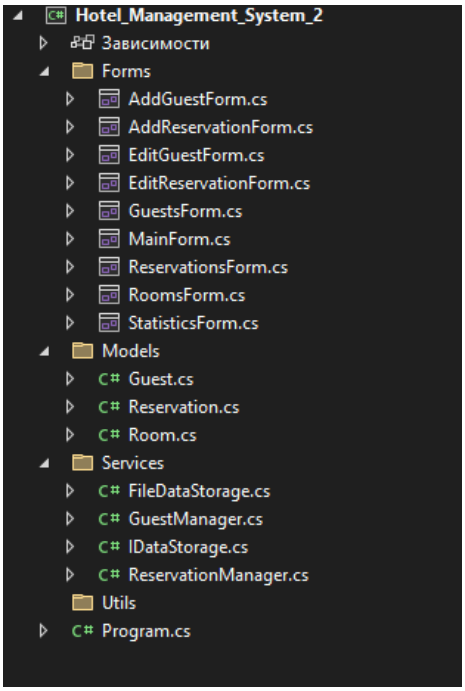

Fig.12. class diagram

4.6. Demonstration of application functionality

The developed application was tested by performing core operations through the user interface. The screenshots below demonstrate the results of completing different operations.

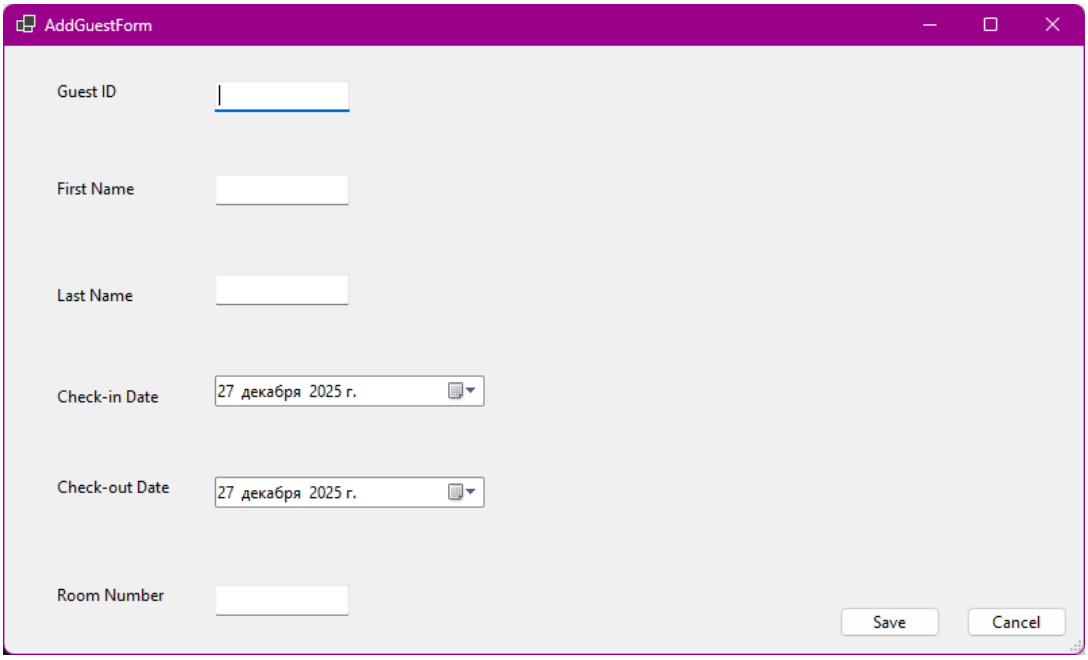

Fig.13. Adding a new guest

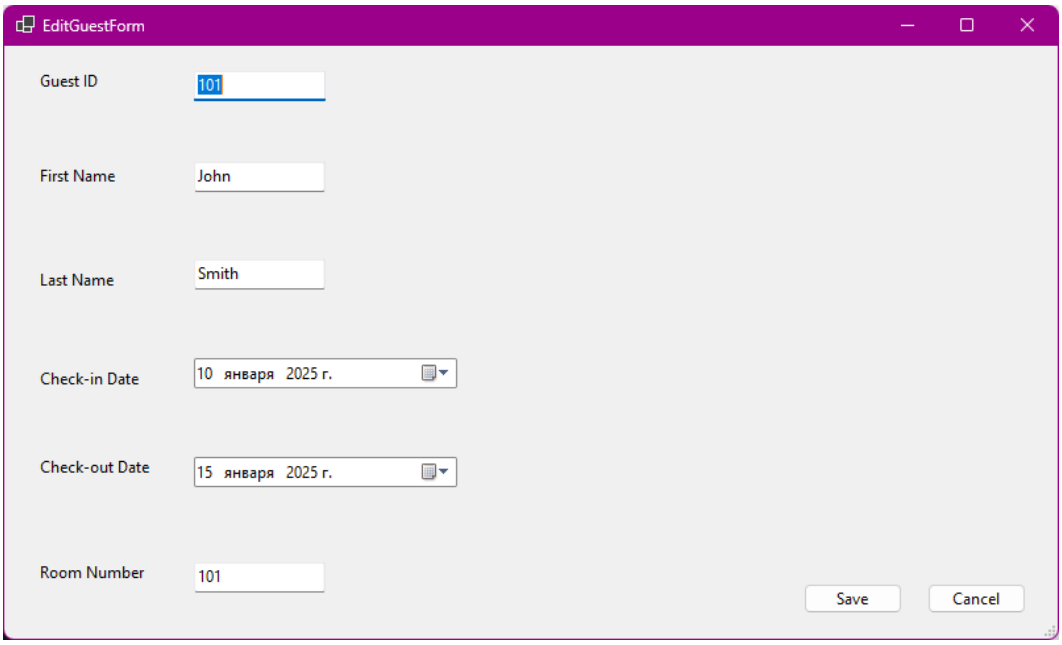

Fig.14. Editing guest information

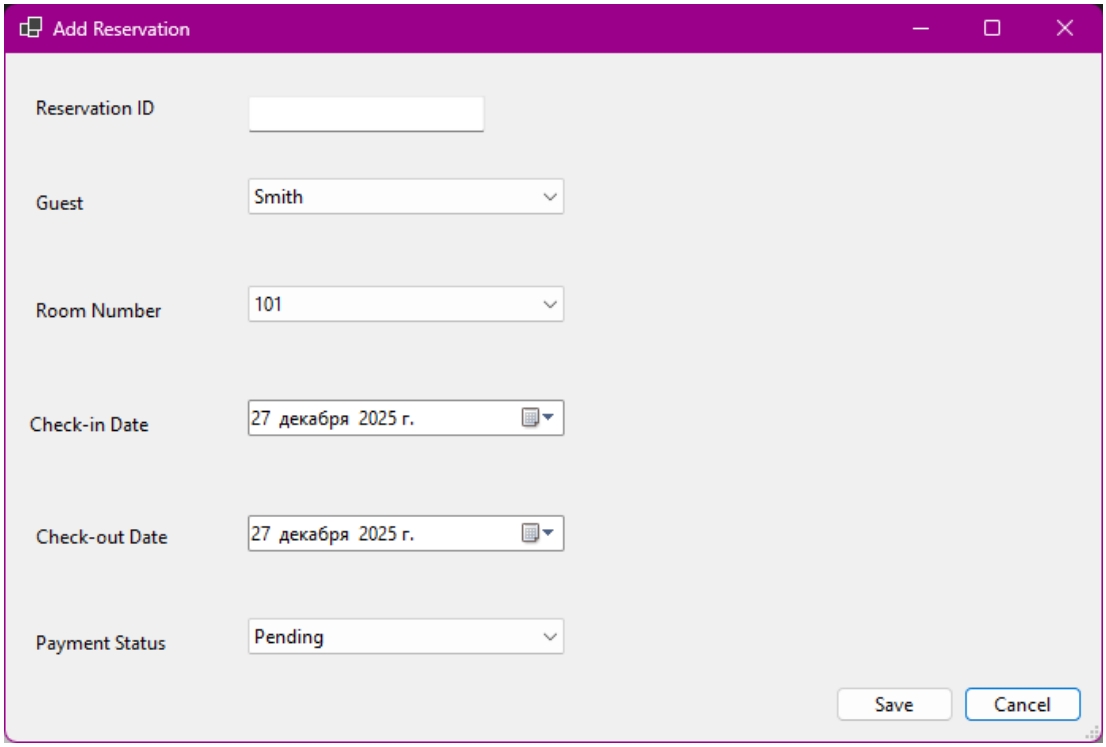

Fig.15. Creating a reservation

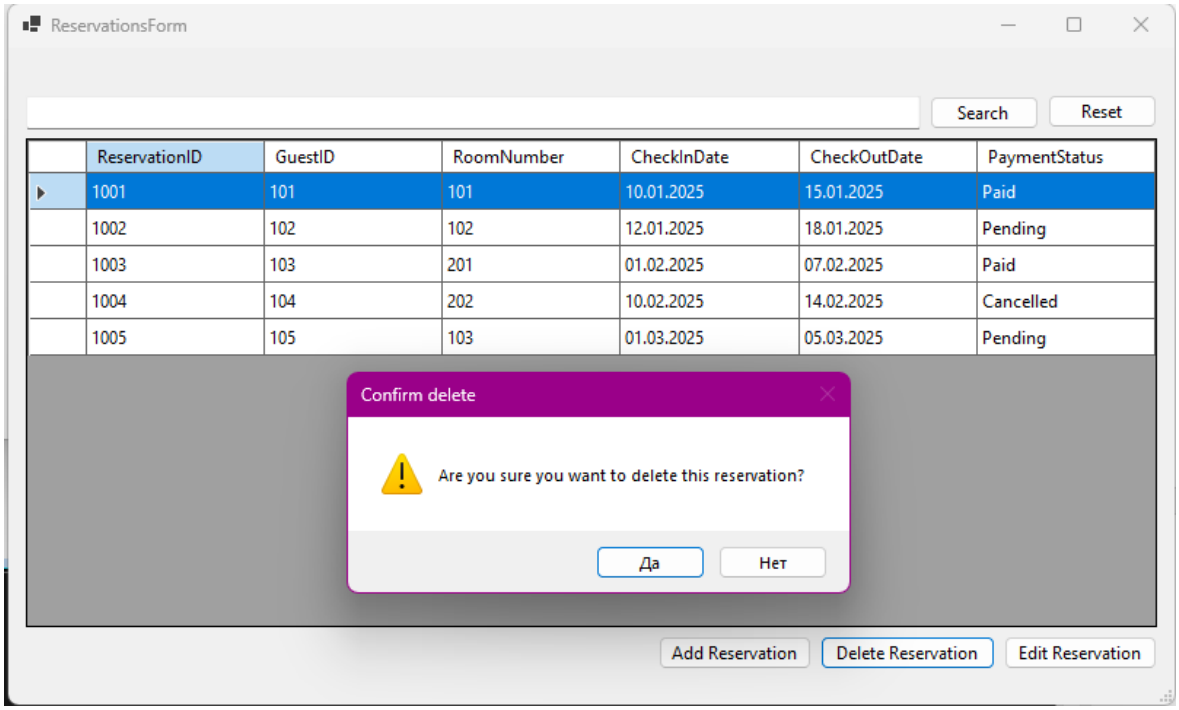

Fig.16. Deleting a reservation

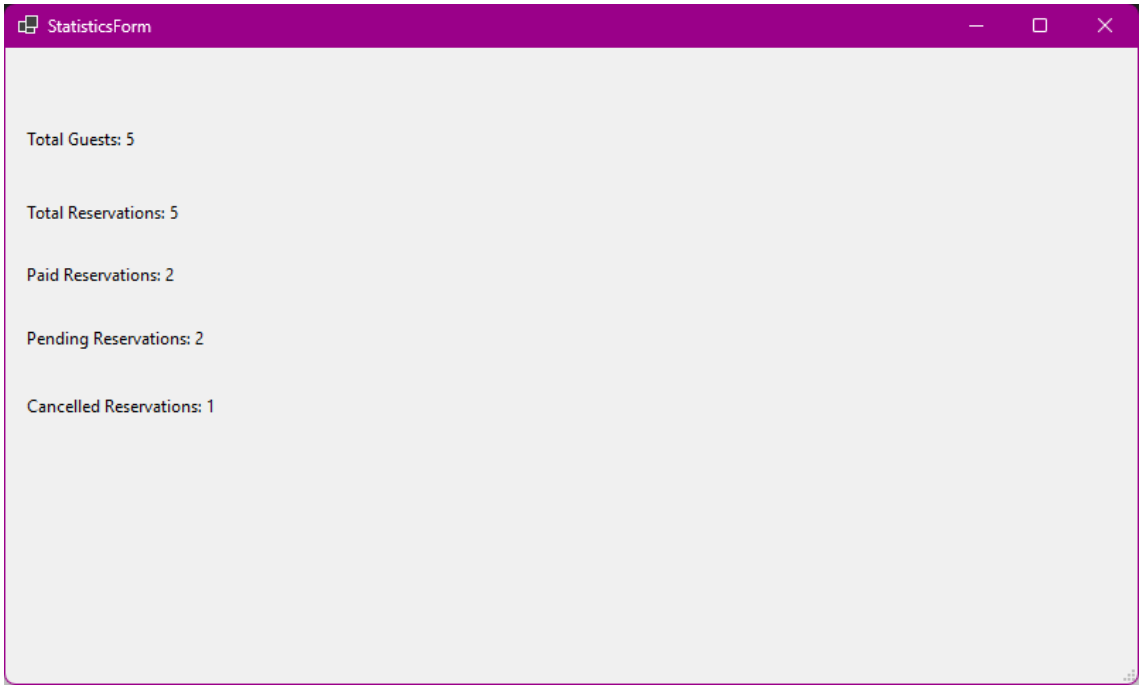

Fig.17. Viewing statistics

5. TESTING

5.1. How testing was organised

Testing of the Hotel Management System was organised as manual functional testing during the development process. Testing was performed directly through the graphical user interface to verify correct interaction between the user interface, business logic, and JSON data storage.

5.2. Scripts

Testing scripts were defined as a sequence of manual actions performed by the user through the application interface. Each script represents a step-by-step procedure to verify a specific system function.

Example test script:

1. Open the application.
2. Navigate to the Guests module.
3. Enter guest data into input fields.
4. Click the Save button.
5. Verify that the guest appears in the list.

5.3. Scenarios

Testing scenarios were prepared for all main system modules to cover typical user interactions.

The following testing scenarios were executed:

- Guest management (adding, editing, deleting guests)
- Room management (viewing rooms and updating room status)
- Reservation management (creating, editing, deleting reservations)
- Input validation (incorrect dates, empty fields)
- Data persistence (loading data from JSON files after restart)

5.4. Test cases

The table below presents selected test cases executed during testing.

Table 4. test cases

Test Case ID	Description	Input Data	Expected Result	Test Result
TC-01	Add new guest	Valid guest data	Guest added successfully	Passed
TC-02	Add guest with empty field	Empty First Name	Error message displayed	Passed
TC-03	Create reservation	Valid dates and room	Reservation created	Passed
TC-04	Invalid date input	Check-out < Check-in	Validation error shown	Passed
TC-05	Load stored data	Restart application	Data loaded from JSON	Passed

5.5. Test results

All executed test cases produced results consistent with the expected behaviour. Functional testing confirmed that the implemented features operate correctly under normal usage conditions and handle invalid input appropriately.

5.6. Found errors and corrections

Several issues were identified and corrected during testing:

- Missing validation for empty input fields was detected and resolved by adding required field checks.
- Incorrect date validation initially allowed invalid reservation dates; this was fixed by implementing date comparison logic.
- JSON data files were not created automatically on first application run; this issue was corrected by adding file existence checks.

After applying corrections, all test cases were re-executed successfully.

CONCLUSIONS

Within the framework of this course project, a desktop-based Hotel Management System was designed and implemented using the C# programming language and object-oriented programming principles. The developed application supports key hotel administration tasks, including guest management, room management, reservation handling, and statistical data viewing.

During the development process, the hotel management domain was analyzed and modelled using key entities such as guests, rooms, and reservations. The application was implemented using a modular architecture that separates data models, business logic, and user interface components. Windows Forms were used to develop the graphical user interface, and JSON files were used to ensure persistent data storage between application sessions.

Several challenges were encountered during development, including implementing reliable input validation, enforcing correct date constraints for reservations, and ensuring consistent data storage using JSON files. These challenges were resolved through incremental development, repeated testing, and refinement of application logic and validation mechanisms.

Overall, the developed Hotel Management System meets the functional requirements defined for the course project and demonstrates a deep understanding of object-oriented software development principles. The application operates reliably under normal operating conditions and provides a clear, structured solution for managing hotel-related data.

REFERENCES

Microsoft (May 26, 2017) *Windows Forms documentation*. Available at: https://learn.microsoft.com/en-us/dotnet/desktop/winforms/

Microsoft Visual Studio Code (November 3, 2021) *Working with JSON*. Available at: https://code.visualstudio.com/docs/languages/json

TutorialsPoint (November 16, 2014) *C# – Basic syntax*. Available at: https://www.tutorialspoint.com/csharp/csharp_basic_syntax.htm

Codex (Medium) (May 18, 2021) *Object-oriented programming principles explained*. Available at: https://medium.com/codex/object-oriented-principles-explained-2d1d4bdd3be7